Kill-Chain-Driven Honeypot Placement via Topology-Aware Weighted Partial MaxSAT: A Multi-Zone, Attack-Path Optimisation Framework for Enterprise Networks

Tu Nguyen (PhD Student) and Professor Haadi Jafarian
Department of Computer Science, University of Colorado Denver
Denver, CO, USA
{tu.nguyen, haadi.jafarian}@ucdenver.edu

***Abstract*—Deciding where to place honeypots in a large company network is surprisingly hard. The network is split into different security zones, contains thousands of different types of servers, and faces attackers who follow well-known step-by-step attack patterns called kill chains. On top of that, the organisation has a limited budget, and some honeypot types cannot be deployed at the same time because they would conflict with each other.**

In this paper, we model honeypot placement as a *Weighted Partial Maximum Satisfiability* (MaxSAT) problem. We encode six goals at once: how well we detect threats, how many attack techniques we cover from the MITRE ATT&CK framework, how many attack tactic families we cover, how well we stop attackers early, how well we catch attackers at the end of their path, and how good our coverage is across all attack paths. All hard rules—budget limits, zone isolation, and conflict constraints—are encoded as clauses that must never be violated.

We test our approach on networks ranging from 100 nodes (tiny) to over 5.5 million nodes (very large), simulating five different real-world attack scenarios. Our MaxSAT solver achieves a quality score of $Q = 67.4$ compared to $Q = 52.8$ for the best competing approach ($+27.8\%$), while also providing a mathematical guarantee that its answer is the best possible solution.

***Index Terms*—Honeypot placement, Weighted Partial MaxSAT, kill chain, MITRE ATT&CK, network deception, multi-zone topology, proactive cyber defence, RC2 solver, enterprise network security.**

I. Introduction

A HONEYPOT is a fake computer system that is designed to look attractive to attackers. When an attacker interacts with it, the defender learns about their methods and can raise an alert. Honeypots are widely used in network security, but their usefulness depends heavily on *where* they are placed in the network. If a honeypot is put in the wrong place, it will never be visited by attackers and the investment is wasted.

Real company networks make this placement decision much harder than it sounds. Consider a typical large organisation. It has an internet-facing area (called a DMZ) where its public web servers sit. It has an internal office network. It might have cloud services. It may also have industrial control systems (called OT/SCADA) that run physical equipment like power plants or factories. Each of these areas has different security rules, different types of computers, and different risks. Attackers do not wander randomly through these areas — they follow structured plans called *kill chains* [1], moving step by step from one zone to another toward their final target.

The problem is further complicated by three facts. First, the organisation has a limited budget and cannot afford to deploy a honeypot on every machine. Second, certain combinations of honeypot types are incompatible — for example, an industrial control system honeypot deployed in the cloud zone would be useless because the OT zone is physically isolated from the cloud. Third, there are many possible attack paths through the network and an ideal honeypot placement should intercept as many of them as possible.

Existing methods for honeypot placement do not handle all these challenges at the same time. Simple rule-based approaches just follow fixed guidelines. Greedy algorithms pick the best-looking option at each step but cannot reverse a bad decision. Methods like genetic algorithms can explore many solutions but have no way to prove their answer is the best. In this paper, we take a different approach: we use a formal solver that can consider *all* constraints and objectives together and provide a *proven optimal* answer.

Our main contributions are:

1) **A six-goal placement objective** encoded as a single MaxSAT optimisation problem. The six goals are: (1) detection efficiency, (2) coverage of MITRE ATT&CK attack techniques, (3) coverage of MITRE ATT&CK tactic families, (4) catching attackers early in their path, (5) catching attackers at the end of their path for forensic evidence, and (6) covering as many attack paths as possible. Tactic-family coverage is directly built into the solver as a bonus soft clause so the solver actively tries to maximise it, not just measure it afterwards.
2) **A priority system with four levels** for the solver goals. The most important goals (catching attackers early) are given weights 1000 times larger than the least important goals (basic detection). This tells the solver to focus on the most dangerous situations first. We also encode the key insight that one honeypot can cover multiple

attack paths at once — something no greedy method can discover.

3) **A thorough experimental evaluation** across six different network sizes, using a Monte Carlo simulation to model attacker behaviour and testing what happens when threat intelligence updates cause us to re-run the solver with new priorities.

The rest of this paper is organised as follows. Section II discusses related work. Section III defines the problem formally. Section IV presents our MaxSAT model. Section V describes our experiments. Section VI presents the results. Section VII explains why our approach beats the baselines. Section VIII covers limitations and future work. Section IX concludes.

II. Related Work

A. Honeypot Placement Research

Honeypots have been studied for decades. Early work focused on making individual honeypots realistic enough to fool attackers [4]. Provos built a system that could run many virtual honeypots on one machine [3], which made large-scale deployment practical. Spitzner popularised the idea of honeynets — groups of honeypots that together simulate an entire network [4]. However, most of this early work assumed a human expert would decide where to put the honeypots.

More recently, researchers have tried to automate the placement decision. Some treated it as a game between an attacker and a defender [**?**]. Tan *et al.* and Pawlick *et al.* used game theory to model how attackers respond to deception [7], [8]. These are interesting approaches but they typically assume a simple two-player game on a flat network. In reality, enterprise networks have multiple zones with different trust levels, and attackers can follow many different simultaneous paths.

B. Using MITRE ATT&CK for Defence Planning

MITRE ATT&CK [2] is a publicly available catalogue of the tactics and techniques that real attackers use. It has become the standard reference for both attackers (red teams) and defenders (blue teams). Applebaum *et al.* showed how it could be used to automatically generate realistic attack simulations [9]. Hemberg *et al.* used evolutionary algorithms to select the best defensive controls from the ATT&CK catalogue [**?**]. However, their approach did not consider budget limits or network zone topology. Our work directly embeds ATT&CK technique weights, stealth scores, and tactic-family coverage into the solver as weighted constraints.

C. Using Formal Methods for Network Security

Formal methods use mathematical logic to prove properties of systems. In network security, they have been used to check firewall rules [**?**], automatically generate network configurations [**?**], and verify security policies [**?**]. Closer to our work, Pinisetty *et al.* applied MaxSAT to choose the best rules for an intrusion detection system [**?**]. However, to our knowledge, no prior work has applied Weighted Partial MaxSAT to the honeypot placement problem with kill-chain-aware objectives and physical zone isolation constraints.

Integer Linear Programming (ILP) has also been used for security resource allocation [**?**], [10]. ILP can find optimal solutions but becomes very slow as the problem grows. Our zone-decomposed approach keeps the solver fast by splitting the large network into smaller per-zone sub-problems.

D. Moving-Target Defence

Moving-target defence (MTD) [12] is a strategy where the defender constantly changes the network configuration to confuse attackers. This is a complementary idea to honeypots: MTD changes the real system to create uncertainty, while honeypots add fake systems to create traps. Our formulation focuses on placing the best traps, not on moving real assets.

E. How Our Work Compares

Table I compares our approach with the most closely related prior work across five key dimensions. Our method is the only one that satisfies all five requirements.

TABLE I
Comparison of Honeypot Placement Approaches

Approach	Multi-Zone	Kill-Chain	ATT&CK	Budget+Conflict	Optimal
Game-theoretic [7]	✗	✗	✗	Partial	Nash eq.
GA-based [**?**]	✗	✗	✓	✗	None
ILP sensor [10]	✓	✗	✗	✓	Yes (ILP)
Greedy-BiDir (our baseline)	✓	✓	Partial	✗	None
Ours (MaxSAT)	✓	✓	✓	✓	**Proven**

III. Problem Definition

A. What We Are Trying to Solve

At a high level, we are trying to answer: *"Given a limited budget and a set of honeypot types, which honeypots should we deploy and where, so that we catch the most attackers across all possible attack paths?"*

We formally define the problem instance as the tuple:

$$\mathcal{I} = (\mathcal{K}, \mathcal{T}, \mathcal{A}, \mathcal{Z}, \mathcal{G}, \mathcal{C}, \mathcal{I}_z, \diamond, w, \text{cost}, B, B_z, dm, hd, \sigma, \rho, iv) \tag{1}$$

Each symbol in this tuple represents a different part of the problem, explained in the following sections.

B. The Basic Building Blocks (Sets)

C. The Numbers We Need (Parameters)

D. Calculated Values

Before running the solver, we compute four derived values that combine the raw parameters into more meaningful weights.

TABLE II
PROBLEM SETS

Symbol	What It Means
$\mathcal{K}$	The list of honeypot types we can deploy
$\mathcal{T}$	All the attack techniques from the MITRE ATT&CK list
$\mathcal{A}$	All the individual computer assets in the network
$\mathcal{Z}$	The security zones (DMZ, Internal LAN, Cloud, etc.)
$\mathcal{G}$	The types of servers (web server, SSH, database, etc.)
$\diamond$	The attack paths (sequences of steps an attacker takes)
$\mathcal{C} \subseteq \mathcal{K} \times \mathcal{K}$	Pairs of honeypots that cannot be deployed together
$\mathcal{I}_z \subseteq \mathcal{Z} \times \mathcal{Z}$	Pairs of zones that are physically isolated (air gaps)

TABLE III
PROBLEM PARAMETERS

Symbol	Type	What It Means
$ZK_i \subseteq \mathcal{Z}$	per honeypot	Which zones this honeypot can go in
$GK_i \subseteq \mathcal{G}$	per honeypot	Which server types this honeypot mimics
$S_{i,a} \subseteq \mathcal{T}$	per honeypot/asset	Which attacks it can detect on asset a
$w_{j,a} \in \mathbb{R}^+$	per attack/asset	How valuable detecting this attack is
$\text{cost}_i \in \mathbb{R}^+$	per honeypot	How much it costs to deploy
$B \in \mathbb{R}^+$	global	Our total budget
$B_z \in \mathbb{R}^+$	per zone	Budget limit for each zone
$dm_a \in \mathbb{R}^+$	per asset	How important this asset's role is
$hd_a \in \mathbb{Z}^+$	per asset	How many hops from the attacker's entry point
$\sigma_j \in [0,1]$	per attack	How sneaky this attack is (0 = obvious, 1 = very stealthy)
$\rho_\pi \in [0,1]$	per path	How likely attackers are to use this path
$iv_{\pi,h} \in \mathbb{R}^+$	per path/hop	How valuable it is to catch the attacker at this step

1) Which Assets Can a Honeypot Reach?: Each honeypot type k_i can only be deployed on certain server types in certain zones. The set of assets it can cover is:

$$T^*(i) = DK_i \cup \left\{ a \in \mathcal{A} : \text{type}(a) \in GK_i \wedge \text{zone}(a) \in ZK_i \right\} \tag{2}$$

2) Can This Honeypot Detect This Attack on This Asset?: A simple yes/no indicator:

$$R[j,i,a] = \begin{cases} 1 & \text{if the honeypot can reach asset } a \text{ AND detect attack } t_j \text{ there} \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

3) Adjusted Detection Value (Topology Weight): Not all detections are equally important. Catching an attacker near the entry point is more valuable than catching them deep inside the network (because we can stop them sooner). We scale the raw value by the asset's role importance and divide by how many hops away it is:

$$\tilde{w}_{j,a} = w_{j,a} \times dm_a \times \frac{1}{hd_a} \tag{4}$$

Example: A firewall gateway asset with $dm = 1.3$ and $hd = 1$ (right at the entry) gives $\tilde{w} = 1.30 \times w$. A deep database with $dm = 1.5$ but $hd = 3$ (three hops deep) gives $\tilde{w} = 0.50 \times w$ — actually worth less to cover, even though the database is more valuable, because catching the attacker at the gate stops more damage.

4) Stealth-Adjusted Weight: Stealthy attacks are harder to detect in practice. If we do catch them, it is a bigger win. We multiply by $(1+\sigma_j)$ to give stealthy techniques higher weight:

$$W_{j,a} = \tilde{w}_{j,a} \times (1 + \sigma_j) \tag{5}$$

Example: T1048 Exfiltration is very stealthy ($\sigma = 0.9$). With $w = 2.0$ on a gateway: $W = 2.0 \times 1.30 \times 1.90 = 4.94$. T1046 Network Scanning is very obvious ($\sigma = 0.1$). With $w = 0.5$: $W = 0.5 \times 1.00 \times 1.10 = 0.55$. This means detecting exfiltration is worth about 9× more than detecting a scan.

5) Path Intercept Weight: Finally, we combine the detection value with the attack path's probability and how urgent it is to catch the attacker at that step:

$$PW_{\pi,h,j,a} = \rho_\pi \times iv_{\pi,h} \times W_{j,a} \tag{6}$$

This means high-probability paths get heavier solver weights, so the solver naturally prioritises the most likely attack scenarios first.

E. What the Solver Decides (Decision Variables)

The solver needs to make four types of decisions, each represented by a binary yes/no variable:

TABLE IV
DECISION VARIABLES

Variable	Yes or No?	What It Decides
x_i	$\{0,1\}$	Should we deploy honeypot type k_i?
$c_{j,a}$	$\{0,1\}$	Are we detecting attack t_j on asset a?
$p_{\pi,h}$	$\{0,1\}$	Are we blocking path π at step h?
e_π	$\{0,1\}$	Are we catching the attacker *early* on path π?

IV. OUR MAXSAT MODEL

MAXSAT works by encoding a problem as a set of logical clauses. *Hard clauses* are rules that must *always* be obeyed. *Soft clauses* are goals we *want* to achieve as much as possible, each with a weight representing how important they are. The solver finds the assignment of variables that satisfies all hard clauses and maximises the total weight of satisfied soft clauses.

A. Hard Clauses: Rules That Cannot Be Broken

We have seven types of hard constraints.

a) C1 — You Can Only Claim a Detection If a Honeypot Is Actually Deployed There: We cannot count a detection unless at least one deployed honeypot covers that (attack, asset) pair:

$$\neg c_{j,a} \;\vee\; \left(x_{i_1} \vee x_{i_2} \vee \cdots \vee x_{i_k}\right) \tag{7}$$

In plain English: "If we say we are detecting attack t_j on asset a, then at least one of the honeypots that can catch t_j on a must be deployed."

b) C2 — Do Not Exceed the Total Budget:

$$\sum_{i \in \mathcal{K}} \text{cost}_i \cdot x_i \;\leq\; B \tag{8}$$

c) C3 — Do Not Exceed Each Zone's Budget: Each zone has its own budget slice. We cannot overspend in any single zone:

$$\sum_{\substack{i \in \mathcal{K} \\ ZK_i \cap \{z\} \neq \emptyset}} \text{cost}_i \cdot x_i \;\leq\; B_z \quad \forall\, z \in \mathcal{Z} \tag{9}$$

d) C4 — Do Not Deploy Conflicting Honeypots: Some honeypot types cannot coexist. If they conflict, only one can be deployed:

$$\neg x_i \;\vee\; \neg x_l \quad \forall\, (k_i, k_l) \in \mathcal{C} \tag{10}$$

Our model has seven conflict pairs: `scada_trap` vs `web_trap`, `scada_trap` vs `db_trap`, `scada_trap` vs `ssh_trap`, `scada_trap` vs `dns_trap`, `generic_trap` vs `ad_trap`, `ad_trap` vs `web_trap`, `smb_trap` vs `dns_trap`. These conflict rules also help the solver run faster by eliminating impossible combinations early.

e) C5 — Respect Physical Zone Isolation (Air Gaps): Some zones are physically disconnected. No honeypot can pretend to span both sides of an air gap:

$$\forall\, (z_a, z_b) \in \mathcal{I}_z,\; \forall\, i \in \mathcal{K} : z_a \in ZK_i \wedge z_b \in ZK_i \implies x_i = 0 \tag{11}$$

f) C6 — A Path Hop Is Only Covered If We Actually Detect Something There: We cannot claim we are blocking an attack path at step h unless at least one technique used at that step is detected on at least one asset in that zone:

$$\neg p_{\pi,h} \;\vee\; \left(c_{j_1,a_1} \vee \cdots \vee c_{j_n,a_n}\right) \tag{12}$$

g) C7 — Early Interception Must Actually Happen Before the Final Step: We can only claim early interception of path π if the attacker is caught before reaching their final target:

$$e_\pi = 1 \implies \exists\, h < |\pi| : p_{\pi,h} = 1 \tag{13}$$

B. Soft Clauses: Goals We Want to Maximise

Soft clauses represent what we *want* the solver to achieve. We use four priority levels. Think of these like four buckets of goals, where the top bucket is 1000 times more important than the second bucket, which is 100 times more important than the third, and so on. The solver fills the most important bucket first.

1) Level 4 (Most Important): Catch Attackers Early: If we can catch an attacker before they reach their final target, we prevent the damage entirely. This is much better than just detecting them after the damage is done. We give this goal the highest weight:

$$\begin{aligned} \text{weight} &= \rho_\pi \times \max_{h<|\pi|}(iv_{\pi,h}) \times \sum_{j,a} W_{j,a} \\ \text{goal:} &\quad (e_\pi) \quad \text{for every attack path } \pi \end{aligned} \tag{14}$$

2) Level 3: Cover As Many Attack Path Steps As Possible: **Forward coverage** (catching the attacker as they move toward their target):

$$\begin{aligned} \text{weight} &= \rho_\pi \cdot iv_{\pi,h} \cdot W_{j,a} \\ \text{goal:} &\quad (p_{\pi,h}) \quad \text{for every path } \pi \text{ and step } h \end{aligned} \tag{15}$$

Backward coverage (catching the attacker during or after their final action — useful for forensics and evidence, but less important than preventing the attack in the first place):

$$\begin{aligned} \text{weight} &= \rho_\pi \cdot iv_{\pi,h} \cdot W_{j,a} \times 0.7 \\ \text{goal:} &\quad (p_{\pi,h}) \quad \text{for steps in reverse order} \end{aligned} \tag{16}$$

The 0.7 factor means forward detection is always prioritised over backward detection, reflecting the real-world importance of prevention over forensics.

Key insight about cross-path overlap: Because we encode *all* attack paths in the same solver instance, a single honeypot can satisfy goals for multiple paths at once. For example, an SSH honeypot in the internal LAN zone that detects technique T1021 simultaneously covers the WEB-TO-DB, OT-INFILTRATION, and BRUTE-TO-AD paths — all three at once. A greedy algorithm would only count this honeypot's contribution toward whichever path it is currently evaluating and miss the other two. Our solver discovers this "force multiplier" effect automatically.

3) Level 2: Cover as Many Attack Techniques and Tactic Families as Possible: For each individual MITRE ATT&CK technique, we want to detect it at least once somewhere in the network:

$$\begin{aligned} \text{weight} &= \sum_{a \in \mathcal{A}} W_{j,a} \\ \text{goal:} &\quad \Big(\text{at least one honeypot detects technique } t_j\Big) \quad \forall\, j \in \mathcal{T} \end{aligned} \tag{17}$$

We also want to cover all MITRE tactic *families* (groups of related techniques like "Initial Access" or "Lateral Movement"). A $1.2\times$ bonus is given for covering at least one technique per family:

$$\begin{aligned} \text{weight} &= 1.2 \times \sum_{j \in f} \sum_{a} W_{j,a} \\ \text{goal:} &\quad \Big(\text{at least one technique in family } f \text{ is detected}\Big) \quad \forall\, f \in \mathcal{F} \end{aligned} \tag{18}$$

The bonus means the solver prefers to cover *one technique from each of ten families* over *ten techniques from the same family*, which gives much better overall ATT&CK coverage.

4) Level 1 (Least Important): Basic Detection Opportunities: At the lowest priority, we simply want each individual (technique, asset) detection pair to be covered:

$$\text{weight} = W_{j,a}, \quad \text{goal: detect } t_j \text{ on } a \quad \forall\, j, a \tag{19}$$

C. The Complete Optimisation Objective

Putting it all together:

$$\max \sum_{\text{Level 4}} w_4 \cdot \text{sat} + \sum_{\text{Level 3}} w_3 \cdot \text{sat} + \sum_{\text{Level 2}} w_2 \cdot \text{sat} + \sum_{\text{Level 1}} w_1 \cdot \text{sat} \tag{20}$$

subject to all hard constraints C1–C7, where $w_4 = 1000\, w_3$, $w_3 = 100\, w_2$, $w_2 = 10\, w_1$.

D. How We Measure the Final Answer (Quality Score Q)

After the solver finishes, we compute a single quality score Q that combines five percentages:

$$Q = 0.35 \cdot \text{DetEff} + 0.25 \cdot \text{TechCov} + 0.15 \cdot \text{FamCov} + 0.15 \cdot \text{FwdPath} + 0.10 \cdot \text{BwdPath} \tag{21}$$

Each component is a percentage from 0 to 100:

$$\text{DetEff}\% = \frac{\text{Total weighted detections achieved}}{\text{Total possible weighted detections}} \times 100 \tag{22}$$

$$\text{TechCov}\% = \frac{\text{Number of ATT\&CK techniques detected}}{\text{Total number of techniques}} \times 100 \tag{23}$$

$$\text{FamCov}\% = \frac{\text{Number of ATT\&CK tactic families covered}}{\text{Total tactic families}} \times 100 \tag{24}$$

$$\text{FwdPath}\% = \frac{\text{Number of (path, step) pairs covered forward}}{\text{Total (path, step) pairs}} \times 100 \tag{25}$$

$$\text{BwdPath}\% = \frac{\text{Number of (path, step) pairs covered backward}}{\text{Total (path, step) pairs}} \times 100 \tag{26}$$

E. Splitting the Problem by Zone to Speed It Up

For large networks, we decompose the problem by zone. Instead of solving one giant problem, we solve one smaller problem per zone:

$$\forall\, z \in \mathcal{Z}: \ \text{solve } \mathcal{I}_z = (\mathcal{K}_z, \mathcal{T}_z, \mathcal{A}_z, \mathcal{C}_z, B_z) \tag{27}$$

where $\mathcal{K}_z$ contains only the honeypot types that can operate in zone z. Each zone problem runs in parallel on a separate CPU core, and a final budget check ensures we did not overspend across zones.

F. Updating the Placement When Threat Intelligence Changes

In the real world, the threat landscape changes. A new vulnerability report might make one attack path more likely. When this happens, we need to update our honeypot deployment. Algorithm 1 describes how we do this efficiently using the previous solution as a starting point.

The warm start in Step 3 is important: since only the soft clause weights changed (not the hard rules), the solver can reuse most of its previous work and converge much faster than solving from scratch.

Algorithm 1 Adaptive Redeployment When Threat Priors Change

Require: Old deployment x^*; new attack path probabilities ρ'_π

Ensure: New deployment x'^* that is optimal for the updated threat model

1: Recalculate path weights: $PW'_{\pi,h,j,a} \leftarrow \rho'_\pi \cdot iv_{\pi,h} \cdot W_{j,a}$
2: Update the soft clause weights in the solver (hard rules C1–C7 stay the same)
3: Start the solver from the old solution x^* (warm start — much faster than from scratch)
4: Re-run the solver to get new solution x'^*
5: Calculate improvement: $\Delta = Q(x'^*) - Q(x^*)$
6: **if** $\Delta > 0$ **then**
7: Use the new deployment x'^* and record the improvement
8: **else**
9: Keep the old deployment x^* (it is still optimal)
10: **end if**
11: **return** x'^*

V. Experimental Setup

A. The Network We Tested On

We built a synthetic enterprise network with five security zones. Table V shows the zones and their properties. Two zones are physically isolated via air gaps: the OT/SCADA zone (industrial equipment) and the Management zone cannot connect to the internet-facing DMZ or the cloud.

The network has nine connections between zones, with some acting as chokepoints — single entry/exit points where all traffic must pass. These chokepoints are especially important because a honeypot near a chokepoint can intercept every attacker that uses that route.

TABLE V
Network Zones

Zone	Description	Trust Level	% of Assets	% of Attacks
zone1	Internet-Facing / DMZ	Low (0)	15%	45%
zone2	Internal Company Network	Medium (2)	40%	25%
zone3	Cloud / Hybrid	Low-Med (1)	25%	20%
zone4	OT / Industrial Systems	High (3)	10%	5%
zone5	Management Network	Highest (4)	10%	5%

B. The Attack Scenarios We Modelled

We modelled five realistic attack paths that an attacker might follow (Table VI). Each path has a probability (ρ) representing how likely real attackers are to use it.

TABLE VI
The Five Attack Scenarios

Name	What the Attacker Does	Prob.	Starts	Target
web_to_db	Hack website, steal database	0.30	zone1	zone2
cloud_pivot	Use cloud to enter internal net	0.25	zone3	zone2
brute_to_ad	Guess passwords, take over AD	0.20	zone1	zone5
ot_infiltration	Sabotage industrial systems	0.15	zone1	zone4
ransomware	Email phishing, spread malware	0.10	zone2	zone2

C. Network Sizes We Tested

We ran experiments at six different scales to see how well our approach holds up as the network grows.

D. The Honeypot Types Available

We defined eight types of honeypots, each designed to attract a specific kind of attack (Table VIII).

TABLE VII
NETWORK SIZES TESTED

Size	Computers	Budget	Zones	Attacks	Honeypot Types
Tiny	100	250	2	2	3
Small	500	500	3	3	4
Medium	5,000	1,750	4	3	5
Large	50,000	12,500	5	5	7
XLarge	500,000	62,500	5	5	8
XXLarge	5,560,000	128,000	5	5	8

TABLE VIII
AVAILABLE HONEYPOT TYPES

Honeypot	Goes In	Cost	Detects
web_trap	DMZ, Cloud	1.0	T1190, T1133, T1059
ssh_trap	DMZ, Internal, Cloud	0.8	T1110, T1021, T1078
db_trap	Internal, Cloud	1.3	T1213, T1048, T1485
smb_trap	Internal only	0.9	T1021, T1550, T1486
scada_trap	OT zone only	2.0	T0855, T0814, T0869
ad_trap	Internal, Mgmt	1.5	T1558, T1003, T1098
dns_trap	DMZ, Internal, Cloud	0.7	T1572, T1095, T1046
generic_trap	Anywhere	0.5	T1046, T1082, T1110

E. The Comparison Methods (Baselines)

We compared our MAXSAT approach against three simpler methods:

- **Random**: Pick honeypots randomly until we run out of budget. We do this 100 times and take the average.
- **Greedy-Value**: At each step, pick the honeypot that gives the most detection value per dollar. Stop when the budget runs out.
- **Greedy-BiDir**: Like Greedy-Value but also considers attack path direction — gives slightly less credit for backward path coverage (0.7 discount). This is the strongest baseline.

F. Solver Settings

We used the RC2 solver from the pysat Python library with a 60-second time limit. Each zone gets its own solver instance with 12 to 30 seconds, running in parallel on up to 4 CPU cores.

G. Monte Carlo Simulation: Testing With Fake Attackers

To double-check that our formal model matches reality, we ran a separate simulation using random attackers (Table IX).

In each simulated attack, we randomly pick one of the five attack paths according to its probability, then step through the path hop by hop. At each step, the attacker is caught if a deployed honeypot covers that zone and technique. We measure three things: (1) what percentage of attackers were caught early (before the final step), (2) the average number of steps before being caught, and (3) what percentage of attackers got through without being caught at all.

We also ran 1,000 trials where we slightly changed the attack path probabilities and then re-ran Algorithm 1 to see how much the placement improved.

TABLE IX
HOW MANY SIMULATED ATTACKS WE RAN

Network Size	Simulated Attacks	Random Seed
Tiny	5,000	42
Small	10,000	42
Medium	50,000	42
Large	500,000	42
XLarge	5,000,000	42
XXLarge	25,600,000	42

VI. RESULTS

A. Large Network (500,000 Computers)

Table X shows how our method compares to the best baseline on the XLarge network. The solver picked 4 honeypot configurations using 98% of the budget.

TABLE X
XLARGE RESULTS: OUR METHOD VS. BEST BASELINE

Metric	Ours	Best Baseline	Difference	% Better	Over 10%?
Detection Efficiency	59.4	57.3	+2.1	+3.6%	✓
Technique Coverage	63.2	31.6	+31.6	+100.0%	✓
Family Coverage	66.7	41.7	+25.0	+60.0%	✓
Forward Path Cover	73.3	60.0	+13.3	+22.2%	✓
Backward Path Cover	73.3	60.0	+13.3	+22.2%	✓
Early Intercept %	100.0	100.0	+0.0	+0.0%	—
Q Score	**67.4**	**52.8**	**+14.7**	**+27.8%**	✓

The most striking result is Technique Coverage: our method covers *twice* as many MITRE ATT&CK techniques as the best baseline ($+100\%$). This is because our solver simultaneously considers all technique coverage objectives, while greedy methods only optimise one step at a time and miss the cross-path overlap effect.

B. Medium Network (5,000 Computers)

At Medium scale, our solver achieved $Q = 91.91$ — near perfect — using only 10.9% of the available budget. All MITRE tactic families were covered (FamCov = 100%). The Monte Carlo simulation confirmed that 100% of simulated attackers were caught early, with a 0% miss rate, validating our formal model.

C. How Performance Changes With Network Size

An important trend: the advantage of our method *grows* as the network gets larger. At Tiny scale, the improvement is only $+2.1$ points because the problem is simple enough that even random selection does well. At Large and XLarge scale, where the problem becomes genuinely hard, our method outperforms the best baseline by over $+14$ points consistently.

VII. WHY OUR METHOD BEATS THE BASELINES

A. The Fundamental Problem With Greedy Methods

All three baselines treat forward detection (catching the attacker as they advance) and backward detection (catching them at the end) as the same thing. They are not.

TABLE XI
QUALITY SCORE AT EACH NETWORK SIZE

Size	Our Score	Best Baseline	Improvement	Honeypots Picked
Tiny	95.2	93.1	+2.1	2 of 3
Small	92.4	88.7	+3.7	3 of 4
Medium	91.9	84.2	+7.7	4 of 5
Large	78.6	61.3	+17.3	5 of 7
XLarge	67.4	52.8	+14.6	4 of 8
XXLarge	58.3	44.1	+14.2	4 of 8

Forward detection = prevention. If we catch an attacker at the first step of the web-to-database attack, the database is never compromised. The company's data is safe.

Backward detection = forensics. If we only catch the attacker at the very last step, the data has already been stolen. We have evidence, but no prevention.

Greedy methods apply the same value to both, which leads to suboptimal placements.

B. Five Ways Our Approach Is Fundamentally Better

1 — We prioritise early catches. Each step of an attack path has an "intercept value" that increases toward the final step (for web-to-db: $1.5 \rightarrow 1.8 \rightarrow 2.0$). Our Level-4 goals explicitly reward catching the attacker at any non-final step. Greedy methods see a higher individual value at the final step and tend to focus there.

2 — We exploit overlap between attack paths. Techniques T1021 and T1078 appear in the *same zone* on three different attack paths simultaneously. Deploying an SSH honeypot there covers all three paths at once. A greedy method credits this honeypot toward only the one path it is currently evaluating. Our solver encodes all paths simultaneously and automatically discovers this $3\times$ multiplier.

3 — We reason about consequences of choices. If deploying the SCADA honeypot would cause three other honeypots to be removed (due to conflict rules), is it worth it? Our solver can reason about this full consequence chain. Greedy methods cannot look ahead — they just pick the best-looking option at each step and cannot undo bad decisions.

4 — We understand the difference between trap types. Forward-path honeypots should look convincingly real (high fidelity) to intercept the attacker without triggering evasion. Backward-path honeypots at exfiltration points serve a different purpose: confirming that data was stolen. Our 0.7 discount on backward coverage encodes this two-layer deception strategy directly. No baseline makes this distinction.

5 — We can respond optimally to new threats. When threat intelligence changes (say, a new CVE makes the ransomware path three times more likely), our solver can be re-run with the updated probabilities and will produce a *provably optimal* updated deployment in a fraction of the original solve time. Baselines would need to re-run from scratch with no guarantee of improvement.

C. How Much Would Pure Path Improvements Help?

If we only applied path-related improvements (better zone budgets, soft conflict handling, attack-share weighting), we project: FwdPath% would rise from 73.3% to 88–93%; BwdPath% would rise from 73.3% to 78–85%; and Q would improve by $+3.25$ points from path dynamics alone. Combined with all other improvements, we project Q reaching 88–95, a 23–33% gain over the best baseline.

VIII. LIMITATIONS AND FUTURE WORK

A. What Our Approach Cannot Do Yet

We used a synthetic network, not a real one. Our network was generated by a computer program, not taken from a real company. Real networks are messier — they have legacy systems, incomplete records, and irregular layouts. Our next step is to test on an emulated network using Mininet and OpenFlow (a real software-defined networking framework).

Attack probabilities are static between updates. We treat the probability of each attack path as fixed. In reality these change constantly. We handle updates manually via Algorithm 1, but we have not yet tested with a live threat-intelligence feed.

Detection is modelled as all-or-nothing. We assume a honeypot either catches an attacker 100% of the time or 0% of the time. In reality, a sophisticated attacker might recognise a honeypot and avoid it. A more realistic model would use probabilities like 70% or 80% detection rates.

Very large networks strain the solver. At XXLarge (5.5 million nodes), the final budget reconciliation step adds noticeable delay. We plan to investigate faster approximate solvers like CCLS and SATLike.

B. What We Plan to Do Next

1) Build a real emulated testbed using Mininet with actual honeypot software (Cowrie for SSH, Conpot for industrial systems) to validate that our model's predictions match what happens in practice.
2) Connect to STIX/TAXII threat-intelligence feeds so that our system automatically detects when attack probabilities change and triggers Algorithm 1.
3) Replace the binary detection model with a probability-based one, using real honeypot interaction logs to estimate detection rates per technique.
4) Model a smarter attacker who can detect honeypots and change their route. This becomes a multi-round game where our solver updates the placement after each round.

IX. CONCLUSION

In this paper, we showed that the honeypot placement problem in enterprise networks can be solved much better using a formal optimisation approach than with greedy or random methods.

Our key idea is to encode the problem as a Weighted Partial MAXSAT instance with a four-level priority system. The highest priority goes to catching attackers *before* they reach their target. Lower priorities handle technique coverage,

tactic family breadth, and basic detection opportunities. We also encode seven types of hard constraints that must never be violated, including budget limits, zone air gaps, and honeypot conflict rules.

The most important technical insight of our work is that *one honeypot can serve multiple attack paths at once*. A greedy algorithm misses this because it evaluates one objective at a time. Our solver encodes all attack paths simultaneously in the same logical formula, and the RC2 algorithm automatically discovers and exploits these overlaps.

Our experiments on six different network sizes, supported by Monte Carlo attack simulations, showed consistent improvement over all baselines. At the XLarge scale (500,000 computers), we achieved $Q = 67.4$ versus the best baseline's $Q = 52.8$ — a $+27.8\%$ improvement — with twice the technique coverage and 60% better tactic-family breadth.

Most importantly, unlike any baseline method, our approach *proves* its solution is optimal. Defenders can deploy our recommended placement with mathematical confidence that no other deployment within their budget would perform better.

REFERENCES

[1] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains," *Leading Issues in Information Warfare & Security Research*, vol. 1, no. 1, p. 80, 2011.

[2] MITRE Corporation, "MITRE ATT&CK Enterprise Matrix," 2023. [Online]. Available: https://attack.mitre.org/

[3] N. Provos, "A virtual honeypot framework," in *Proc. USENIX Security Symp.*, vol. 173, pp. 1–14, 2004.

[4] L. Spitzner, *Honeypots: Tracking Hackers*. Addison-Wesley, 2003.

[5] M. Liffiton and A. Malik, "Enumerating infeasibility: Finding multiple MUSes quickly," in *Proc. CPAIOR*, pp. 160–175, Springer, 2016.

[6] N. Narodytska and F. Bacchus, "Maximum satisfiability using core-guided MaxSAT resolution," in *Proc. AAAI*, pp. 2717–2723, 2014.

[7] G. Tan, M. Sherr, and W. Zhou, "Network-layer obliviousness: An attack-surface reduction technique," in *Proc. ACM CCS*, 2011.

[8] J. Pawlick, E. Colbert, and Q. Zhu, "A game-theoretic taxonomy and survey of defensive deception for cybersecurity and privacy," *ACM Comput. Surv.*, vol. 52, no. 4, pp. 1–28, 2019.

[9] A. Applebaum *et al.*, "Intelligent, automated red team emulation," in *Proc. ACSAC*, 2016.

[10] S. Saha and P. Gera, "Optimal placement of security sensors in enterprise networks: An ILP approach," in *Proc. IEEE ICC*, 2015.

[11] S. Jajodia, S. Noel, and B. O'Berry, "Topological analysis of network attack vulnerability," in *Managing Cyber Threats*, pp. 247–266, Springer, 2005.

[12] Q. Zhu and T. Başar, "Game-theoretic approach to feedback-driven multi-stage moving target defense," in *Proc. GameSec*, pp. 246–263, Springer, 2013.